POLY PMNA · DIPLOMA CURRICULUM REV2021

Course 2131 | Semester 2 | Theory | Shared handbook

Problem Solving and Programming

Build systematic problem-solving ability and transferable programming fundamentals through algorithms, control flow, modularity, data handling, testing and debugging.

Course code	**2131**
Revision	**REV2021**
Semester	**Semester 2**
Type	**Theory**
Catalogue scope	**9 catalogue programme assignments**

COURSE ORIENTATION

How to use this handbook

This page is a structured learning aid for the shared catalogue record. Study the concept, complete the applied activity, retain the evidence, and compare the result with the latest approved programme instructions. Where the official syllabus differs, the official record takes precedence.

Source and verification note: The course identity, code, semester and subject type are taken from the tracked POLY PMNA REV2021 catalogue, which indexes the official SITTTR scheme. The official course-content reference is SITTTR course 2131; the scheme index is SITTTR REV2021 diploma syllabus. The direct subject-content page was not machine-readable or returned an unavailable-file response during preparation, so module headings and explanatory teaching material on this page are an original study-handbook structure, not claimed verbatim SITTTR wording. Confirm current hours, marks, credits, outcomes and assessment against the latest official programme record before examination use.

Learning outcomes

- Explain the central concepts and vocabulary of the subject using clear technical language.
- Translate a problem into an algorithm, implementation or practical experiment and test it with meaningful cases.
- Create readable code or a laboratory record with algorithm, output, test evidence and reflection.
- Identify assumptions, limitations and common errors, then improve the work using feedback.

Shared-record context

This code is assigned to 9 catalogue programme assignments in the tracked catalogue. The page therefore focuses on common learning practice and avoids claiming programme-specific technical details that were not verified.

Study rule: Do not treat the author-created examples as official examination questions or official mark distribution.

STUDY MAP

Modules and evidence

Author-created study map; verify official hours and marks			
Module	**Heading**	**Purpose**	**Evidence to retain**
1	Problem-solving cycle and algorithms	Decompose a problem, identify inputs and outputs, and express a solution before writing code.	Convert a word problem into a flowchart, pseudocode and a hand-worked trace table.
2	Programming foundations and control flow	Use the official language confirmed for the programme to represent data and control execution.	Design a menu-driven calculation programme with valid-input and invalid-input paths.
3	Functions and modular design	Break a program into small, testable functions with clear interfaces and documentation.	Refactor one long algorithm into functions and write a test table for each function.
4	Collections, text and structured data	Process collections and text while avoiding indexing, mutation and data-validation errors.	Trace a search and a sort on a small data set, then identify the boundary cases.
5	Testing, debugging and integrated problems	Use evidence to locate faults and solve a small integrated programme-relevant problem.	Submit a tested mini-program with problem statement, algorithm, test evidence and reflection.

MODULE 1

Problem-solving cycle and algorithms

Purpose-led study **Apply + reflect**

Decompose a problem, identify inputs and outputs, and express a solution before writing code.

Core topics−

- Problem definition and assumptions
- Decomposition and pattern recognition
- Pseudocode, flowcharts and trace tables
- Correctness, termination and complexity intuition

Study method

Read the concept, connect it to a diploma-level situation, make a labelled record or diagram, and check the result against the stated outcome. Keep programme-specific technical decisions within the approved project, laboratory or workplace context.

Applied activity

Convert a word problem into a flowchart, pseudocode and a hand-worked trace table.

> **Evidence:** Keep the completed activity, a short reflection and any source or test record in your course folder.

Common mistakes and corrections−

- Starting with implementation before defining the problem, inputs, outputs or acceptance criteria.
- Making a claim without a source, measurement, test or documented observation.
- Using a template mechanically instead of adapting it to the approved programme context.

Correction: state the assumption, choose evidence, perform the check, and record the decision.

Self-check−

1. Explain the main idea of this module in your own words.
2. Apply it to a small technical or workplace scenario.
3. Identify one likely failure and describe the evidence that would diagnose it.

MODULE 2

Programming foundations and control flow

Purpose-led study | Apply + reflect

Use the official language confirmed for the programme to represent data and control execution.

Core topics−

- Variables, data types, expressions and input/output
- Sequence, selection and nested selection
- Loops, counters, accumulators and sentinels

- Validation and boundary conditions

Study method

Read the concept, connect it to a diploma-level situation, make a labelled record or diagram, and check the result against the stated outcome. Keep programme-specific technical decisions within the approved project, laboratory or workplace context.

Applied activity

Design a menu-driven calculation programme with valid-input and invalid-input paths.

> **Evidence:** Keep the completed activity, a short reflection and any source or test record in your course folder.

Common mistakes and corrections−

- Starting with implementation before defining the problem, inputs, outputs or acceptance criteria.
- Making a claim without a source, measurement, test or documented observation.
- Using a template mechanically instead of adapting it to the approved programme context.

Correction: state the assumption, choose evidence, perform the check, and record the decision.

Self-check−

1. Explain the main idea of this module in your own words.
2. Apply it to a small technical or workplace scenario.
3. Identify one likely failure and describe the evidence that would diagnose it.

MODULE 3

Functions and modular design

Purpose-led study | Apply + reflect

Break a program into small, testable functions with clear interfaces and documentation.

Core topics−

- Parameters, return values and scope
- Decomposition and reuse
- Preconditions, postconditions and comments
- Common interface and state errors

Study method

Read the concept, connect it to a diploma-level situation, make a labelled record or diagram, and check the result against the stated outcome. Keep programme-specific technical decisions within the approved project, laboratory or workplace context.

Applied activity

Refactor one long algorithm into functions and write a test table for each function.

> **Evidence:** Keep the completed activity, a short reflection and any source or test record in your course folder.

Common mistakes and corrections−

- Starting with implementation before defining the problem, inputs, outputs or acceptance criteria.
- Making a claim without a source, measurement, test or documented observation.
- Using a template mechanically instead of adapting it to the approved programme context.

Correction: state the assumption, choose evidence, perform the check, and record the decision.

Self-check−

1. Explain the main idea of this module in your own words.
2. Apply it to a small technical or workplace scenario.

3. Identify one likely failure and describe the evidence that would diagnose it.

MODULE 4

Collections, text and structured data

Purpose-led study | Apply + reflect

Process collections and text while avoiding indexing, mutation and data-validation errors.

Core topics−

- Arrays or lists, indexing and traversal
- Strings and text processing
- Searching and sorting concepts
- Records/objects and file handling only when officially required

Study method

Read the concept, connect it to a diploma-level situation, make a labelled record or diagram, and check the result against the stated outcome. Keep programme-specific technical decisions within the approved project, laboratory or workplace context.

Applied activity

Trace a search and a sort on a small data set, then identify the boundary cases.

> **Evidence:** Keep the completed activity, a short reflection and any source or test record in your course folder.

Common mistakes and corrections−

- Starting with implementation before defining the problem, inputs, outputs or acceptance criteria.
- Making a claim without a source, measurement, test or documented observation.

- Using a template mechanically instead of adapting it to the approved programme context.

Correction: state the assumption, choose evidence, perform the check, and record the decision.

Self-check−

1. Explain the main idea of this module in your own words.
2. Apply it to a small technical or workplace scenario.
3. Identify one likely failure and describe the evidence that would diagnose it.

MODULE 5

Testing, debugging and integrated problems

Purpose-led study **Apply + reflect**

Use evidence to locate faults and solve a small integrated programme-relevant problem.

Core topics−

- Syntax, runtime, logic and data errors
- Test cases, edge cases and assertions
- Debugging hypotheses and minimal fixes
- Integrated numerical, text or record-processing problem

Study method

Read the concept, connect it to a diploma-level situation, make a labelled record or diagram, and check the result against the stated outcome. Keep programme-specific technical decisions within the approved project, laboratory or workplace context.

Applied activity

Submit a tested mini-program with problem statement, algorithm, test evidence and reflection.

Evidence: Keep the completed activity, a short reflection and any source or test record in your course folder.

Common mistakes and corrections−

- Starting with implementation before defining the problem, inputs, outputs or acceptance criteria.
- Making a claim without a source, measurement, test or documented observation.
- Using a template mechanically instead of adapting it to the approved programme context.

Correction: state the assumption, choose evidence, perform the check, and record the decision.

Self-check−

1. Explain the main idea of this module in your own words.
2. Apply it to a small technical or workplace scenario.
3. Identify one likely failure and describe the evidence that would diagnose it.

ASSESSMENT PREPARATION

Evidence, revision and quality gates

Before submission or examination

1. Confirm the current SITTTR/SBTE title, hours, credits, outcomes and assessment.
2. Define the problem or prompt and state assumptions.
3. Show the method, calculation, algorithm, project record or communication structure.
4. Attach test, source, supervisor, workplace or reflection evidence as applicable.
5. Review clarity, safety, ethics, citations, originality and accessibility.

Revision checklist

- Can I define the main terms without copying?
- Can I apply the method to a new situation?

- Can I explain one common error and its correction?
- Can I show evidence for each claimed outcome?
- Can I state the limitations and the next improvement?

```
Evidence record
Subject: 2131 — Problem Solving and Programming
Task or question:
Assumption or requirement:
Method / design / procedure:
Result or artefact:
Test / source / supervisor evidence:
Reflection and next action:
```

SELF-CHECK AND EXAMINATION PRACTICE

Question bank

Recall question

Define two important terms from Problem Solving and Programming and distinguish them.
Answer guidance: define the terms, state assumptions, show the method, use a specific example, and cite or retain evidence where appropriate.

Explain question

Explain why the method in this handbook matters for a diploma student.
Answer guidance: define the terms, state assumptions, show the method, use a specific example, and cite or retain evidence where appropriate.

Apply question

Apply one module method to a realistic technical, project or workplace situation.
Answer guidance: define the terms, state assumptions, show the method, use a specific example, and cite or retain evidence where appropriate.

Evaluate question

Identify a weak approach, state the evidence needed, and propose a defensible improvement.
Answer guidance: define the terms, state assumptions, show the method, use a specific example, and cite or retain evidence where appropriate.

Create question

Design a small artefact, plan, test, report section or presentation that demonstrates the subject outcomes.
Answer guidance: define the terms, state assumptions, show the method, use a specific example, and cite or retain evidence where appropriate.

REFERENCES AND GLOSSARY

Reference trail

1. Official SITTTR REV2021 diploma syllabus catalogue.
2. Official SITTTR course-content reference for code 2131.
3. POLY PMNA Study Hub, for the current revision index and related lesson pages.

Glossary

Terms used in this handbook	
Term	**Working meaning**
Outcome	A capability the learner should demonstrate, not just a topic read.
Evidence	A record, artefact, result, source, test or review that supports a claim.
Acceptance criterion	A condition used to decide whether a requirement or deliverable is satisfactory.
Reflection	A brief evidence-based account of what worked, what did not and what should change.

Prepared as a POLY PMNA study handbook. Always follow the latest official SITTTR/SBTE instructions for the programme and examination session.